* Unit-5 *
Tractable & Intractable Problems

* **Tractable Problem**:- Problem which can be solved in polynomial time or there is an efficient algorithm that solves it in polynomial time.

Polynomial time

$O(n)$, $O(n^2)$, $O(n^3)$

Fast & efficient.

⇒ Sorting (merge sort)
Searching (Binary Search)

* **Intractable Problem**:- Problem that can't be solved in polynomial time or there is no efficient algorithm to solve it in polynomial time.

Time complexity $O(2^n)$, $O(n!)$

very slow

⇒ Traveling Salesman Problem (TSP)
Hamilton path.

* Complexity Classes :-

(a) **P-Class** :- Problem that can be solved in polynomial time.

Ex: Sorting & searching.

-> P-class easy to solve & easy to verify.

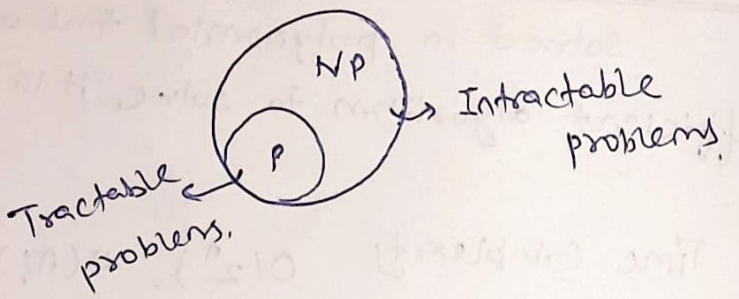
(b) **NP-Class** :- Cannot be solved in polynomial time but can be verified in polynomial time.

(using Non-deterministic algorithm)

Ex: 0/1 Knapsack problem, Su-du-ku, TSP etc.

Note $P \subseteq NP$

means all P-class are NP class.



* Reduction

Suppose 2 problem A & B

If we convert A into B (with in polynomial time)

$A \propto B$ (A is reducible to B)

means if B is solvable that A is also solvable

Property

-> if A is reducible to B, B is in P then A in P ($A \propto B$, $B \in P \Rightarrow A \in P$)

-> A is not in P implies B is not in P.

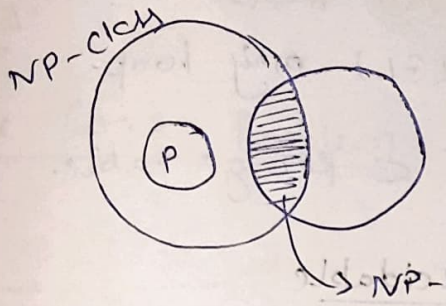
* **NP-Hard :-** A problem is NP-hard if every problem in NP can be polynomially reduced to it.

It is at least as hard as NP-class.

Ex Halting Problem.

* **NP-complete :-** A problem, if it is in NP & it is also in NP-hard.

Intersection of NP & NP-hard.



Ex
- Hamilton Path
- Vertax Cover.

Relation b/w NP, P, NP Hard & NP-complete

* **Decidability & Undecidability :-**

→ A problem can be decidable (Solvable) if

There exists a Turing Machine (TM) which always Halt at each input.

& gives Answer in Yes/No.

(No infinite loop) → Recursive language.

Ex $L = \{ w | w$ has equal number of a's & b's$\}$

Properties
- always terminate
- Correct result.

→ Undecidables problems if there is no TM exist that halt on each input.
or gives correct output.
either infinite loop or Solution doesn't exist.

Eg: Halting Problem.
PCP (Post Correspondance Problem)
TM equivalence Problem.

* (Non-Solvable)

Note: Decidable always halts whereas Recognizable (REL) may loop.

Decidable ⊂ Recognizable.

* How to Prove Undecidable

Usually use Reduction
Known undecidable problems (like Halting Problem)

Logic
If known undecidable → reduce to new problem
= New problem also undecidable.

~~Crook~~

* Cook's Theorem
* Every NP problem can be reduced to SAT problem
SAT is first NP-Complete Problem.
each NP problem can be converted into SAT problem

SAT => Boolean Sactisfiability Problem.

$(x_1 \vee x_2) \wedge (\neg x_1 \vee x_3)$

on which variable expression is true.

Cook's if a NP problem,
solvable by TM (Nondeterministic)

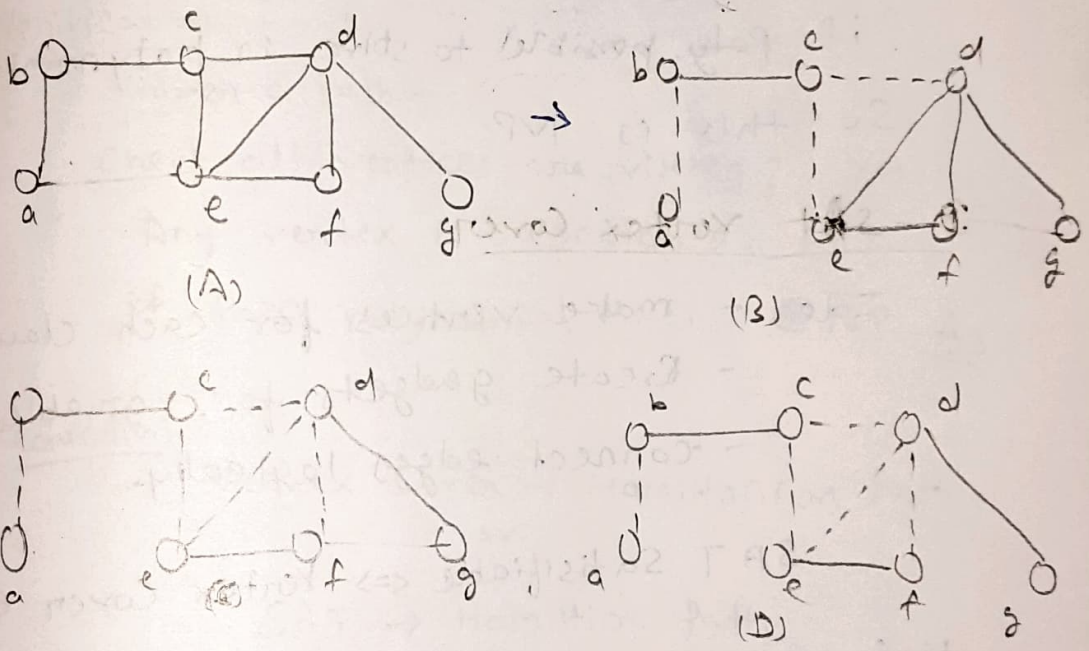
then

1. Represents in computation of TM
2. convert each steps in boolean variable
3. convert whole computation into Boolean formula.

So TM accepts input <=> Boolean Formulas satisfiable.

* Problems

(1) Vertex Cover Problem :-



given graph G = (V, E)

A vertex cover is a set of vertices such that ~~each edges lays~~ at least on endpoint of each edges lays in that set.

Ex
Vertices = {A, B, C}
Edges = {(A, B), (B, C)}

vertex cover
{B} covers both edges.

Given G & K
If there a vertex cover of size ≤ k ?

=> It is a complete-NP problem.

Verification
Given a set S:
check each edge should be covered or not ?
size ≤ k ?
if ~~poly~~ possible to solve in polynomial time
So this is NP.

3 - SAT vertex Cover

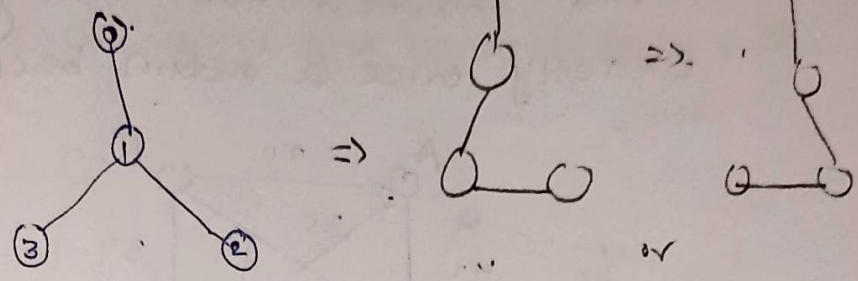
Idea - make vertices for each clause
- Create gadgets for variables
- connect edges logically.

SAT Satisifiable <=> Vertex Cover exists.

Use Network monitering
Security Placement
Sensor Placemet.

(2) Hamiltonian Path Problem

→ A path which visits each vertex exactly one time.



path $0 \to 1 \to 2 \to 3$

Hamiltonian Cycle –

Same path but start = end.

Verification

Given a Path –

Check all vertices are visited? Yes

Any vertex is repeated? No

if then polynomial time. $\in NP$ Ans

Reduction

Vertex Cover → Hamiltonian Path

or

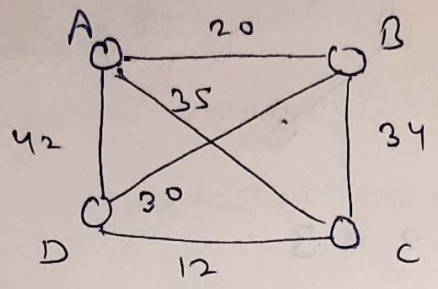
SAT → Hamilton Path.

→ Problem is hard because we must try many permutation.

Eg = Routing planning
Scheduling

(3) Traveling Salesman Problem (TSP)

Given cities
& Distance between cities.
Find shortest tour visiting all cities exactly once & return back.



Choose minimum distance.

Decision version

IS there a tour with cost $\leq k$?

This is complete - NP problem

* Number of paths = $(n-1)!$

very large (exponential)

~~verification~~ Reduction logic

Hamiltonian Cycle → TSP

It uses dijkstra Algorithm.

Assign

edge exist = 1 cost

edge not exist = ∞ cost

(TSP cost $\leq n$)

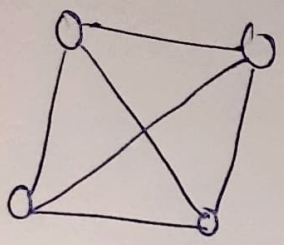
Eg:
- Airline scheduling
- Logistics
- Delievery Routing

~~(4) Ellent~~

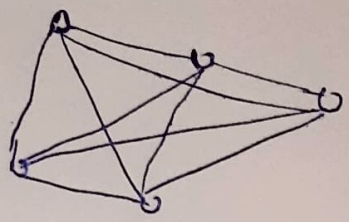
(4) ## Clique Decision Problem :-

In a graph, a clique is a set of vertices such that each vertex are connected to each other.

3 Clique



4 Clique.



5 – Clique

Clique = Complete subgraph.

Problem statement

Given a graph G & integer k,
Does G contain a clique of size $\geq k$?

Verification

Given a set S:

Check size = k

each pair connected. ?

time. $O(k^2)$

Polynomial time

∴ So Clique ∈ NP

Eg:-
- Social networks (Group detection)
- Bioinformatics
- Pattern Matching